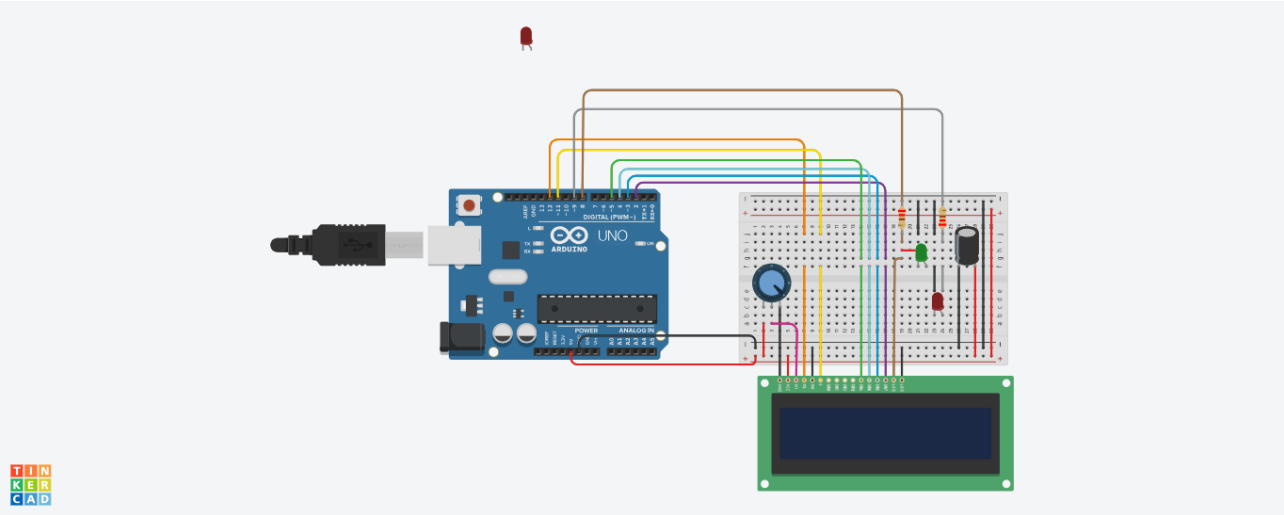
PROJECT PROGRESS & PRESENTATION GUIDE

Optimizing Energy Consumption of Industrial Robots through Lightweight Deep Q-Learning

Mayinja Joel
BSc Computer Science · Uganda Christian University
Registration: S24B23/047

A presentation-ready account of the research problem, systematic review, chosen dataset, software and physical prototypes, current progress, and next steps.



Repository Tinkercad circuit illustration.

Prepared from the project repository, the proposal (Chapters 1–3), and the revised systematic literature review. Implemented code is distinguished from planned hardware and unverified results.

1. The project in one minute

Industrial robots can draw substantial energy during repeated acceleration and movement. For Ugandan small and medium-sized manufacturers, energy cost, unreliable electricity, and expensive computing create a combined barrier to automation. This project investigates whether a learned controller can move a simulated robot arm more efficiently, whether a smaller student can imitate it, and whether that student can eventually run on low-cost edge hardware.

Two complementary sides

Side	Contribution	Current boundary
Software and simulation	Python/Gymnasium/PyBullet robot environment, DQN teacher code, and a browser dashboard for energy and task metrics.	The interface exists; Windows dependency installation stops while compiling PyBullet, so live Flask training/results are not verified.
Physical-facing demonstrator	Arduino/Tinkercad illustration communicates an energy state with LEDs and an LCD; future work may add a servo.	Educational mock only—not the trained DQN, an industrial robot, or a calibrated energy measurement.

Presentation promise

“I have built the foundations of a simulation-based research prototype: dataset exploration, robot-environment code, the teacher-training pipeline and a dashboard interface. Next I will complete setup, generate reproducible results, align implementation with the proposal, then build the distilled and physical demos.”

Do not present the proposal’s 20% saving, 85% student retention, 100 ms Raspberry Pi latency, or safe operation during outages as achieved results. These are targets, not measurements.

2. Background, problem, and research gap

The proposal frames the challenge as a triple constraint: conventional movement can waste energy; advanced learning controllers can require costly computing; and conventional control does not adapt when electricity weakens or fails. Improving motion rather than buying new robot hardware is the initial research focus.

Why this matters in Uganda

- Manufacturers need dependable energy for production; outages and voltage fluctuations increase operational risk and generator costs.
- A small, efficient controller could make automation more accessible if it meets performance and safety requirements.
- This is a research prototype and evidence-building exercise—not a deployed factory product.

What the systematic review found

The revised review (12 October 2026) follows PRISMA reporting. Its Scopus searches displayed 226 records across overlapping search strings; after 31 duplicates, 195 records were screened. Fifty full texts were assessed and six primary studies met the final criteria. One included study reported 23.21% energy reduction with parallel DRL trajectory planning; DQN studies focused on positioning/navigation rather than energy, while KD examples focused on robotic perception. The review found no included study combining energy-aware DRL control, policy distillation and low-cost deployment.

The contribution is a testable combination of ideas, not a claim that the gap is already solved. The proposal asks whether energy-aware DQN control, model compression and edge deployment can work together under simulated grid instability.

Sources: Mayinja Joel, Systematic Literature Review, second draft (12 October 2026), abstract and PRISMA methods; proposal Chapters 1–2.

3. Dataset: what it is and why it was chosen

The repository includes notebooks/steel_data.csv, a steel-industry energy-consumption time series. The exploratory notebook checks types and missing values, parses date/time, groups observations by load type and hour, and plots consumption patterns.

Observed in supplied CSV	Value
Rows and fields	35,040 observations × 11 columns
Coverage / sampling	1 Jan–31 Dec 2018; readings every 15 minutes
Missing cells	0 in this CSV snapshot
Load labels	Light_Load 18,072 · Medium_Load 9,696 · Maximum_Load 7,272
Example variables	Usage_kWh, reactive power, CO_2, power factor, NSM, weekday, Load_Type

Why this dataset fits the study

- It supplies an industrial energy context rather than a generic household electricity example.
- Its load labels and time patterns give a starting point for exploring demand and motivating light/heavy phases.
- Its 15-minute observations can inform exploratory plots; they are not robot-control measurements.

Important limitation: the CSV records steel-plant electrical consumption; it contains no KUKA joint torque, velocity, trajectory or robot success labels. The current simulator uses seeded synthetic targets. The CSV is explored separately and is not currently fed into the DQN training loop. Any mapping from three industrial load categories to two simulated task phases must be justified and validated.

For the final report, add the dataset's formal source/citation, license, cleaning procedure and rationale for how (or whether) load categories calibrate the simulator.

4. Software prototype: what exists

Component	Current implementation	Status / caveat
Robot environment	Gymnasium/PyBullet KUKA iiwa; 23-value observations; 400-step episodes; two seeded light/heavy target phases; 50 Ws buffer; 3.0 Ws peak threshold.	Implemented in code. Energy and buffer are simplified estimates, not power-meter readings or a physical battery.
Teacher agent	DQN with two 256-unit hidden layers; replay buffer 10,000; batch 64; epsilon-greedy selection.	Pipeline is written; actual training is unconfirmed because PyBullet installation failed.
Actions	25 choices: no movement plus four non-zero speed levels for each of six controlled joints.	KUKA model has seven physical joints; joint 6 is locked. Actions are velocity commands, not torque commands.
Baseline / metrics	Fixed-speed inverse-kinematics baseline; grid draw, peak, buffer, reward, task result and learning loss.	Reduced-speed baseline and unstable-grid scenarios in the proposal are not yet in the current runner.
Web dashboard	Responsive Overview, Live monitoring, Data detection and Model management sections.	Static preview works on port 8000; training needs Flask API.

How the present simulation estimates energy

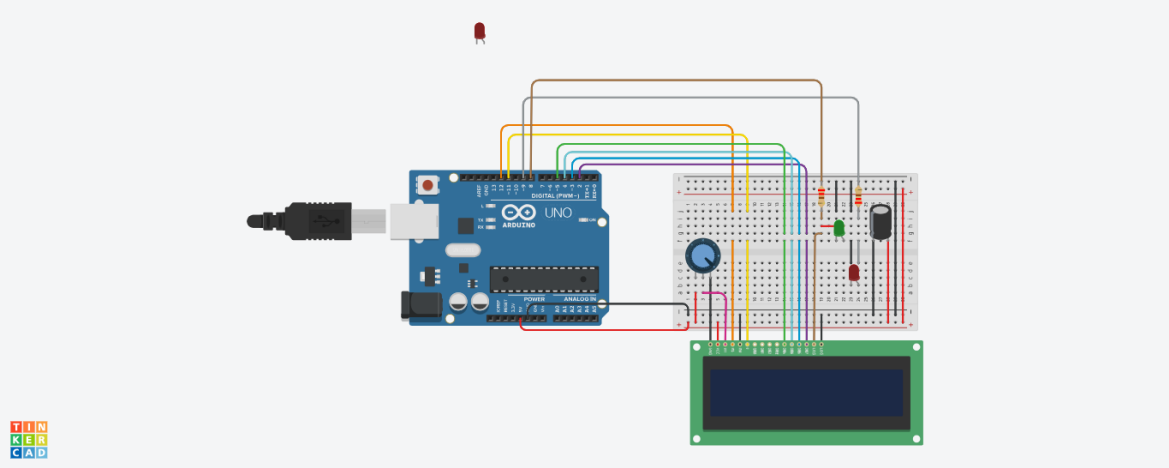
The current environment sums |torque × angular velocity| × time step over six controlled joints. A simple buffer is charged in the light phase and used in the heavy phase. The proposal describes a richer model (copper-loss and idle terms) and a four-state Markov grid model; these remain implementation work and must be reconciled.

Dashboard note: http://127.0.0.1:8000 serves the web page only. Live status and training require Flask at http://127.0.0.1:5000. The latest shared terminal log stops at “Microsoft Visual C++ 14.0 or greater is required”; requirements installation therefore failed before Flask was available.

5. Physical-facing prototype: show it honestly

The physical side is a small educational demonstrator that communicates “save energy in an easy phase, use stored energy in a harder phase.” It does not claim to run the DQN on a robot.

The Arduino sketch currently reads an analog value at A0, compares it with a threshold, and updates a red LED, green LED and 16×2 LCD. Tinkercad notes describe a proposed next version with a servo as a robot-joint stand-in, an LDR for available surplus and a potentiometer for buffer charge. The stored sketch is simpler than that plan.



Repository Tinkercad circuit illustration. Verify that wiring and sketch match before describing it as a working demo.

Part	Demo role	Reality check
Arduino Uno	Reads an input and applies transparent threshold logic.	Runs the Arduino sketch only; not PyTorch or trained DQN.
LEDs + LCD	Display a simple simulated energy state.	Current code does not validate real grid quality.
Optional servo	Moves an arm-like pointer/linkage to show actions.	Visual stand-in, not a KUKA torque-controlled joint.
Optional LDR + pot	Simulate surplus availability and buffer state.	Proxy inputs, not mains sensors or calibrated storage.

For a safe presentation, use low-voltage USB/Arduino power only. Do not connect this demo to mains or an industrial robot. The Raspberry Pi 4 and ONNX student controller are a separate future hardware-in-the-loop objective.

6. Evaluation plan and targets—not results

Question	Planned measurement	Proposal target
RQ1 · Does teacher control reduce energy?	Energy per completed task, success and peak; matched tasks/seeds.	≥20% less energy than constant-speed baseline; task completion no more than 5 percentage points lower.
RQ2 · Can student retain performance?	Model size, success, energy-saving retention, action agreement, Pi latency.	<500,000 parameters; retain ≥85% of teacher performance/saving; median Pi decision ≤100 ms.
RQ3 · Does it handle variable supply?	Safety, completion and energy across stable, brownout, outage/UPS and solar-surplus scenarios.	At least 20% less energy per task than non-adaptive baseline while maintaining safe operation.

Recommended experimental discipline

- Use identical task placements, grid sequences and evaluation seeds for controllers; keep training and test seeds separate.
- Report energy per completed task with success rate. Saving energy by failing tasks is not success.
- Repeat experiments and report spread and confidence intervals. The proposal calls for paired Wilcoxon tests and Holm correction.
- Measure both inference-only and round-trip Pi latency; use a USB power meter if measuring Pi draw.
- Label every value as simulated, physical-demo, or measured hardware.

Current measured performance: no completed training metrics are confirmed in the shared progress. Do not present empty dashboard values as findings or claim hypotheses are proven.

7. Progress to date and next sequence

Workstream	Evidence	Current status
Research foundation	Proposal Chapters 1–3 and revised systematic review.	Research question and study plan documented.
Dataset exploration	35,040-row CSV and Jupyter notebook checks/plots.	Exploratory; not robot joint data or connected to training.
Simulation / DQN	Environment, agent and training runner in src/.	Code present; runtime and quantitative outcomes need verification.
Dashboard	Responsive multi-section web UI in web/index.html.	Static preview confirmed at localhost:8000; API not verified.
Physical demo	Tinkercad illustration and Arduino threshold/LCD/LED sketch.	Simple educational mock; evaluated hardware prototype not shown.
Student / Pi / grid study	Specified as later proposal objectives.	Not completed in the current prototype.

Next steps in order

- Complete Windows C++ Build Tools setup; activate .venv310; install requirements; verify Flask, PyBullet and PyTorch imports.
- Start Flask with `python -m src.dashboard`; check /api/status before training.
- Run 1–3 episodes, verify logs and saved outputs, then run unit tests. Do not start 500 episodes until the short run works.
- Reconcile proposal/code differences in network size, action timing, energy model, baselines, task and grid scenarios with the supervisor.
- Complete repeated teacher experiments; then implement student/ONNX/Pi and unstable-grid evaluation.
- Build and photograph the low-voltage Arduino demonstrator; update sketch/notes to match components actually wired.

The latest shared install attempt failed at PyBullet compilation because Microsoft Visual C++ 14.0+ was unavailable. The requirements install stopped before Flask was installed in that environment.

8. Edit the dashboard and make it yours

In VS Code, open the project folder `C:\\Users\\JOEL\\Desktop\\DQN\\Energy_consumption_DQN`. Press Ctrl+P and type `web/index.html`. The preview URL is not the editor; this is the source file to change.

Personalization	Where/how
Brand, page title, navigation	Search for WATTWISE, the HTML <title> or navigation button labels in web/index.html.
Introduction and school details	Edit the Overview hero and top-bar text. Keep claims consistent with the approved proposal.
Colors, spacing, mobile layout	Edit CSS variables at the start of the <style> block (such as --teal and --canvas) and its @media rules.
Photo or project image	Copy a JPG/PNG into web/assets/. Replace the hero SVG with <img class="hero-photo" src="/assets/your-image.jpg" alt="Describe the image"> and add CSS sizing.
New menu/page/feature	Add a nav button with data-view-target="your-view", a matching section id="view-your-view", then implement and test the feature.
Training API	Edit Flask endpoints/controller in src/dashboard.py; preserve the API paths used by web/index.html.

The repository now has a web/assets/ folder and a Flask route for /assets/... files. For static preview, from the repository root run `python -m http.server 8000 --directory web` and open `http://127.0.0.1:8000`. Live controls need the Flask app after dependencies install.

Tip: keep a copy before big edits, use descriptive image alt text, and refresh with Ctrl+R. The static server serves files only; it cannot train the model.

9. Suggested presentation talk track

Opening. Good morning. My project is titled “Optimizing Energy Consumption of Industrial Robots through Lightweight Deep Q-Learning.” I am investigating how a robot controller might complete tasks while using energy more carefully, and whether a compact controller could eventually operate on low-cost hardware.

Problem and motivation. Industrial arms repeat movements many times. Speed and acceleration affect energy demand, while SMEs can also face unreliable electricity and limited access to expensive computing. My proposal therefore connects energy-aware robot control, model compression and operation under simulated grid conditions. The prototype is not yet a factory deployment.

Literature and gap. In my systematic review I screened literature using a PRISMA-guided process. Six primary studies met the final criteria. The studies covered energy-efficient control, DQN applications, or knowledge distillation, but the review found no included study combining all three with low-cost deployment. That gap informs my proposed teacher–student approach.

Dataset. I chose the steel-industry energy dataset because it provides industrial load context. The supplied CSV contains 35,040 quarter-hourly observations across 2018, with light, medium and maximum load labels. I have explored its time and load patterns. It is not robot telemetry, so I am not claiming that the DQN learns robot motion directly from this CSV; connecting these patterns to simulation remains future calibration work.

Prototype. The repository contains a Gymnasium/PyBullet KUKA environment, DQN training pipeline and web dashboard. The model has seven physical joints, with six controlled and the wrist locked. The DQN observes 23 values and selects among 25 discrete joint-velocity actions. A simple buffer supports light and heavy phases. A separate Arduino/Tinkercad concept displays a simple energy state; it is not the DQN running on a robot.

Progress and limits. I have implemented foundation code and redesigned the dashboard into Overview, Live Monitoring, Data Detection and Model Management. I can preview the page through the static server. My current setup blocker is installing PyBullet on Windows, which requires Microsoft C++ Build Tools in the environment I tried. I do not yet present trained performance numbers as results. Student, Raspberry Pi and unstable-grid experiments remain future work.

Close. I will complete setup and a short reproducible training test, align the implementation with the proposal, collect repeated baseline comparisons, and then work toward distillation and the low-voltage demo. The outcome will be evidence—not an assumed energy saving—about whether a lightweight controller can reduce energy per completed task while preserving success. Thank you.

10. Likely questions and defensible answers

Question	Suggested answer
Why use a steel-industry dataset?	It provides industrial load context and load labels for exploration. It is not robot joint telemetry; the DQN uses simulated robot observations. Any calibration link needs a transparent method.
Has the controller saved 20% energy?	Not established. Twenty percent is a proposal target. I need successful training and repeated matched-seed comparisons with task completion reported.
Is the Raspberry Pi running the model?	No. Pi deployment and ONNX inference are later objectives. No Pi latency or power result is claimed.
Does the Arduino circuit run the DQN?	No. It is a threshold-based visual demonstrator. The Python/PyTorch teacher and a future ONNX student are separate.
How is energy calculated?	The current simulation sums |joint torque × joint velocity| over a physics step for six joints. It is an estimate; the proposal's richer model needs to be reconciled.
What happens during a grid failure?	The proposal plans normal, brownout, outage/UPS and solar surplus scenarios. The current code does not implement that complete grid model, so I cannot claim outage resilience.
Why DQN?	The controller can be framed as a finite action-choice problem. Discretized joint velocities fit DQN, but are less expressive than continuous-control algorithms.
Main limitation?	Simulation estimates may not transfer to a real robot or factory grid; the task and energy model are simplified. Hardware validation remains necessary.

Revision terms

DQN: Deep Q-Network; learns action values from observations and rewards. Replay buffer: stores experiences for sampled updates. Teacher/student: trained policy and smaller model that imitates it. ONNX: portable model format. Energy per completed task: the central comparison metric, interpreted together with success rate. PRISMA: structured reporting of a systematic review.

Before presenting: identify whether the screen is static preview or live Flask, use screenshots only for existing work, and label future objectives as planned.

11. Proposal-to-code alignment checklist

The proposal is the research plan; the repository describes a narrower, evolving prototype. Reconcile these points with the supervisor before presenting implementation details as final methods.

- Teacher architecture: repository code uses two 256-unit hidden layers; the proposal student table describes a 3×512 teacher with 543,754 parameters. Confirm the approved architecture and keep one source of truth.
- Control action: repository uses 25 one-joint-at-a-time velocity choices; proposal describes speed/acceleration limit choices. Explain and approve the mapping.
- Energy model: current code uses mechanical $|\tau\omega|$ only; proposal includes copper loss and idle power. Upgrade code or formally narrow the model.
- Task: current simulation uses seeded target-reaching phases; proposal specifies pick, grasp, carry and release. Clarify what is implemented versus planned.
- Grid evaluation: current runner measures a constant-speed inverse-kinematics baseline. Proposal includes a reduced-speed baseline and grid scenarios. Implement or justify a scope change.
- Physical/edge scope: Arduino/Tinkercad demo, Raspberry Pi ONNX hardware-in-the-loop, and physical industrial robot are different milestones; report each separately.

Reference documents reviewed

- Mayinja Joel, Optimizing Energy Consumption of Industrial Robots through Lightweight Deep Q-Learning, proposal Chapters 1–3, October 2026.
- Mayinja Joel, Deep Reinforcement Learning and Knowledge Distillation for Energy-Efficient Industrial Robot Control: A Systematic Literature Review, second draft, 12 October 2026.
- Project repository README, source, notebook/CSV, Tinkercad notes and Arduino sketch inspected on 3 November 2026.

This preparation aid does not replace the approved proposal or supervisor guidance. Update it when experiments produce verified results.